

URL Shortener Microservices

Thiết kế, triển khai và vận hành hệ thống rút gọn URL theo kiến trúc Microservices

Đồ án môn học SE361.Q21 — Phát triển phần mềm theo kiến trúc Microservices

University of Information Technology — VNU-HCM

2026

Không phải app CRUD

Một use case đọc rất nhiều:
redirect phải nhanh, ổn
định, không phụ thuộc
analytics.

Microservice đúng nghĩa

Bounded context, database
riêng, API Gateway, HTTP
sync và RabbitMQ async.

Có vận hành

Docker Compose,
healthcheck, Prometheus,
Grafana, smoke test, k6
load test.

Idea bảo vệ: hệ thống nhỏ về nghiệp vụ, nhưng đầy đủ các
vấn đề thật của microservices: chia domain, dữ liệu phân
tán, messaging, cache, resilience, observability và kiểm thử.

Đề cương nhắm tới

DDD & Bounded Context

REST + Async Messaging

Database per Service

Security + Resilience

Docker + Monitoring + Testing

Dự án đã hiện thực

5 binary Go độc lập: gateway, user, url, analytics, notification.

HTTP/JSON qua gateway; RabbitMQ topic exchange cho click, created, deleted, milestone.

4 PostgreSQL database riêng; Redis chỉ làm cache và rate-limit store.

Prometheus/Grafana, circuit breaker, k6 script, smoke/e2e scripts.

Bài toán: URL Shortener không đơn giản như nhìn thấy

Nếu làm kiểu monolith đơn giản

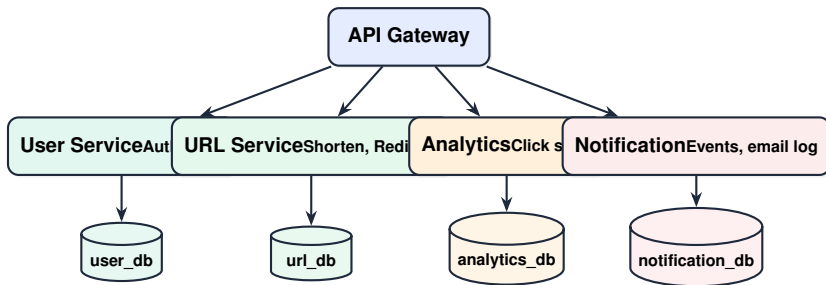
- ▶ Redirect, analytics, notification dính vào cùng request path.
- ▶ Analytics lỗi có thể kéo chậm redirect.
- ▶ Scale phải scale cả khối dù chỉ redirect nóng.
- ▶ DB chung dễ sinh join chéo và coupling schema.

Cách tiếp cận microservices

- ▶ Redirect là hot path, ưu tiên cache và latency.
- ▶ Analytics/notification xử lý bất đồng bộ qua event.
- ▶ Mỗi service sở hữu dữ liệu của mình.
- ▶ Gateway bảo vệ hệ thống bằng auth, rate limit, circuit breaker.

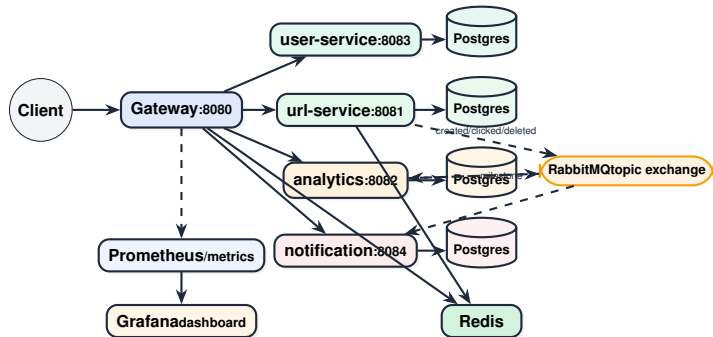
Mục tiêu thiết kế: redirect nhanh khi phụ trợ lỗi một phần, nhưng vẫn thu thập click và gửi thông báo bằng event.

Tách Bounded Context



Điểm cần nhấn mạnh: monorepo chỉ là cách tổ chức source code; ranh giới runtime là binary, container, API và database riêng.

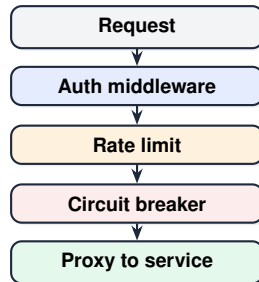
Kiến trúc tổng quan



API Gateway: cửa vào duy nhất

Gateway làm ở edge

- ▶ Reverse proxy route theo path.
- ▶ JWT validation cho route private.
- ▶ Rate limit theo IP bằng Redis.
- ▶ Circuit breaker từng upstream.
- ▶ Gắn X-Correlation-ID.
- ▶ Expose `/metrics` cho Prometheus.



REST API surface

Nhóm	Gateway endpoint	Trách nhiệm service
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/me	Register, login, bcrypt, JWT, profile
URL	POST /api/shorten, GET /api/urls, DELETE /api/urls/{code}	Short code, ownership, deactivate, out-box
Redirect	GET /r/{code}	Redis cache, DB fallback, 301 redirect, click event
Analytics	GET /api/stats/{code}, timeline	Total click, 24h, 7d, referer, time bucket
Notification	GET /api/notifications	List notification theo user, cursor paging

RESTful + pragmatic: route public/private rõ, list API có cursor pagination, limit được clamp, body auth giới hạn 1MB.

URL Service: hot path được tối ưu

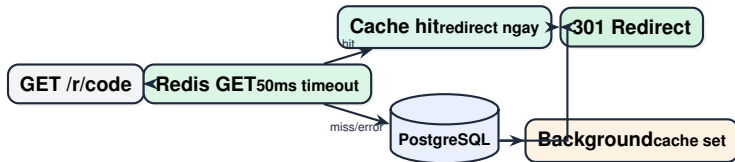
Shorten

- ▶ Validate URL chỉ nhận http/https.
- ▶ Sinh base62 7 ký tự bằng `crypto/rand`.
- ▶ Không gian mã: $62^7 \approx 3.5$ nghìn tỷ.
- ▶ Unique constraint + retry collision.
- ▶ Ghi `url.created` vào outbox trong transaction.

Redirect

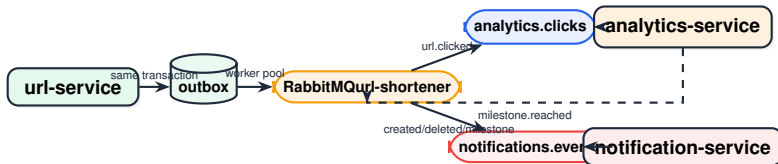
- ▶ Redis read-through cache, timeout 50ms.
- ▶ Redis lỗi/cache miss thì fallback PostgreSQL.
- ▶ Check inactive/expired trả 410.
- ▶ Hash IP, ghi user-agent/referer.
- ▶ Click analytics đi qua event, không block bởi consumer.

Redis: nhanh nhưng không phải source of truth



Câu trả lời khi bị hỏi: Redis mất thì hệ thống chậm hơn nhưng không mất dữ liệu; PostgreSQL vẫn là nguồn dữ liệu đúng duy nhất.

Event-driven: analytics và notification không kéo chậm redirect



Event payload chứa đủ dữ liệu để consumer xử lý, giảm gọi ngược service khác. Đây là trade-off denormalization có chủ đích để giảm coupling.

Outbox Pattern: không để mất event khi broker lỗi

Nếu publish trực tiếp

- ▶ Ghi URL thành công.
- ▶ RabbitMQ timeout đúng lúc publish.
- ▶ DB có dữ liệu nhưng analytics không biết.
- ▶ Dễ mất event, khó reconcile.

Cách dự án làm

- ▶ Ghi URL và event vào outbox cùng transaction.
- ▶ Worker poll mỗi 2 giây, batch 50.
- ▶ 3 worker goroutines.
- ▶ FOR UPDATE SKIP LOCKED tránh đụng nhau.
- ▶ Publish xong mới set published_at.

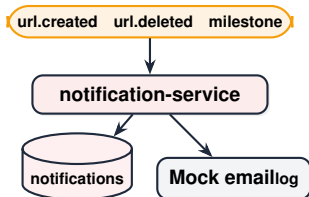
Consumer behavior

- ▶ Consume `url.clicked` từ RabbitMQ.
- ▶ Manual ack/nack.
- ▶ QOS prefetch 1 để kiểm soát áp lực.
- ▶ Invalid JSON hoặc thiếu event id thì ack và discard.
- ▶ Deduplicate bằng bảng `processed_events`.

Stats API

- ▶ Total clicks.
- ▶ Clicks last 24h và last 7d.
- ▶ Top referers.
- ▶ Timeline theo day hoặc hour.
- ▶ Query song song bằng errgroup.
- ▶ Milestone: 10, 100, 1000 clicks.

Notification Service: side effect được cô lập



Điểm nhỏ nhưng nên kể

- ▶ Consumer không làm URL Service chậm.
- ▶ Notification lưu trạng thái `pending`, `sent`, `sent_at`.
- ▶ List notification theo user có cursor pagination.
- ▶ Event `url.created` chứa email để tránh gọi ngược `user-service`.
- ▶ Mock email giúp chứng minh flow side-effect mà không cần SMTP thật.

Security: đủ lớp cần thiết cho đồ án

Identity

Register/login, email unique, password tối thiểu 8 ký tự, bcrypt hash.

JWT

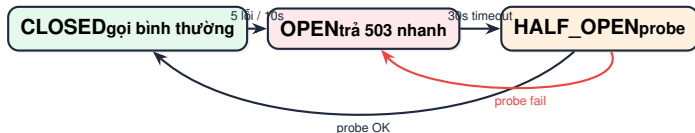
HS256, claims sub/email/iss/iat/exp, verify signing method HMAC.

Data protection

Không lưu raw password, hash IP trước khi đưa vào analytics, ownership check khi delete URL.

Nói thật khi bị hỏi production: demo compose còn hard-code secrets, chưa TLS, chưa refresh token/revocation, chưa RBAC/OAuth2 đầy đủ. Đây là điểm mở rộng, không nên nói quá.

Resilience: khi downstream chết, gateway không chết theo



Circuit breaker: giảm cascading failure, tránh request treo hàng loạt khi service chết.

Rate limit: shorten 10 request/phút/IP, redirect 300 request/phút/IP; Redis lỗi thì fail-open.

Scalability: scale theo nút cổ chai, không scale cả cục

Redirect đọc nhiều

- ▶ Cache Redis giảm query DB.
- ▶ Gateway và url-service stateless ở tầng HTTP.
- ▶ Có thể tăng replica cho gateway/url-service.
- ▶ DB index theo `short_code`.

Async workload

- ▶ Analytics/notification tách khỏi request path.
- ▶ Outbox worker pool 3 goroutines.
- ▶ RabbitMQ durable queue hấp thụ spike.
- ▶ Milestone có unique constraint chống phát nhiều lần.

Cẩn thận khi nói: Docker Compose hiện demo một replica; thiết kế cho horizontal scaling nhưng chưa triển khai Kubernetes/service discovery động.

Database per Service

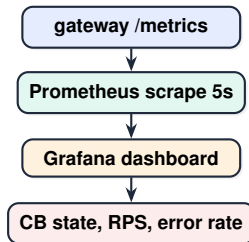
Service	Database	Bảng/ý nghĩa chính
user-service	user_db	users: email unique, password hash, created at
url-service	url_db	urls, outbox; index short_code, user_id, created_at
analytics	analytics_db	clicks, milestones, processed_events dedup
notification	notification_db	notifications: payload JSONB, status, sent at

Luận điểm bảo vệ: không cross-service join, không shared schema. Consistency giữa services đạt bằng event và eventual consistency.

Observability: biết hệ thống đang sống hay đang gãy

Signals đã có

- ▶ Structured JSON logs bằng slog.
- ▶ X-Correlation-ID xuyên gateway.
- ▶ Gateway metrics Prometheus.
- ▶ Grafana dashboard Circuit Breaker Monitor.
- ▶ Health endpoint cho từng service.



Load testing: chứng minh chịu tải và circuit breaker

k6 scenario

- ▶ Start stack: `docker compose up -d -build`.
- ▶ Smoke test health bằng script.
- ▶ k6 tự register user, login, shorten URL.
- ▶ Load test endpoint nóng: `/r/{shortCode}`.
- ▶ Target script hướng tới 10,000 req/s.

Demo failure thật

- ▶ Trong lúc k6 chạy: `docker compose stop url-service`.
- ▶ Gateway nhận lỗi kết nối.
- ▶ Sau 5 lỗi/10s: CB CLOSED -> OPEN.
- ▶ OPEN: trả 503 circuit open ngay, không gọi downstream.
- ▶ Start lại service, 30s sau probe HALF_OPEN.

Benchmark targets trong tài liệu thiết kế

Khu vực	Target	Ý nghĩa
Redirect cache hit	p99 < 5ms	Hot path phải gần như memory lookup
Redirect cache miss	p99 < 20ms	DB fallback vẫn chấp nhận được
Shorten URL	p99 < 50ms	Gồm validate, generate code, insert DB, outbox
Outbox publish latency	< 5s	Eventual consistency có giới hạn
Gateway overhead	p99 < 5ms	Gateway không trở thành bottleneck
Circuit OPEN reject	< 1ms	Shed load nhanh khi downstream lỗi
Stats API	p99 < 30ms	Analytics query có index và chạy song song

Khi bảo vệ nên nói đây là **target/expected benchmark trong dev docs**; kết quả thực tế phụ thuộc tài nguyên máy Docker lúc chạy.

Testing: không chỉ chạy được trên máy em

Unit tests

gateway, events, user,
URL, analytics,
notification.

Smoke/E2E

health ports 8080–8084,
register, login, shorten,
redirect, stats, notification,
delete.

Load test

k6 redirect workload,
monitor circuit breaker
bằng Grafana.

Các case quan trọng: JWT middleware, path rewrite, rate limit, circuit breaker state, invalid event ack/discard, analytics dedup, milestone unique, deleted URL trả 410.

Những thứ nhỏ nhưng thầy rất hay hỏi

- ▶ Có healthcheck từng service/container.
- ▶ Có cursor pagination cho URL và notification.
- ▶ Có clamp limit tối đa 100.
- ▶ Có ownership check khi delete URL.
- ▶ URL inactive/expired trả 410.
- ▶ Redis rate limit fail-open.

- ▶ Event contract đặt trong shared/events.
- ▶ JWT verifier kiểm tra thuật toán HMAC.
- ▶ Analytics dedup event id.
- ▶ Milestone 10/100/1000 chỉ tạo một lần.
- ▶ Logs có service name và correlation ID.
- ▶ RabbitMQ queue durable, exchange topic.

Exactly-once? Không, và đó là câu trả lời đúng

Không hứa quá mức

- ▶ RabbitMQ + outbox thực tế là at-least-once.
- ▶ Message có thể bị redeliver.
- ▶ Network partition không cho exactly-once miễn phí.

Cách giảm tác hại

- ▶ Analytics dùng `processed_events` để dedup.
- ▶ Milestone có unique constraint.
- ▶ Outbox tránh mất event khi broker lỗi.
- ▶ Notification duplicate là hạn chế đã nhận diện.

Câu trả lời mẫu: hệ thống chọn at-least-once delivery + idempotent consumer ở nơi quan trọng, thay vì tuyên bố exactly-once không thực tế.

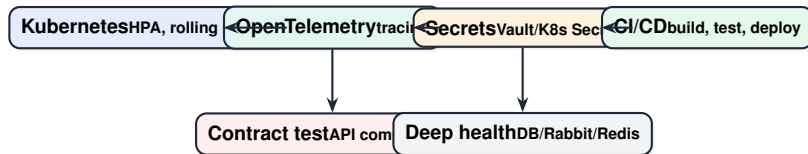
Đã đủ cho đồ án

- ▶ Local Docker Compose chạy toàn stack.
- ▶ Microservice boundary rõ.
- ▶ Có cache, broker, DB riêng, metrics.
- ▶ Có demo failure bằng circuit breaker.

Chưa production-ready hoàn toàn

- ▶ Chưa Kubernetes/service discovery động.
- ▶ Chưa OpenTelemetry tracing/Jaeger.
- ▶ Metrics chủ yếu ở gateway.
- ▶ Secrets demo trong compose, chưa Vault/Secret Manager.
- ▶ Chưa CI/CD workflow và contract test đầy đủ.

Roadmap nếu nâng lên production

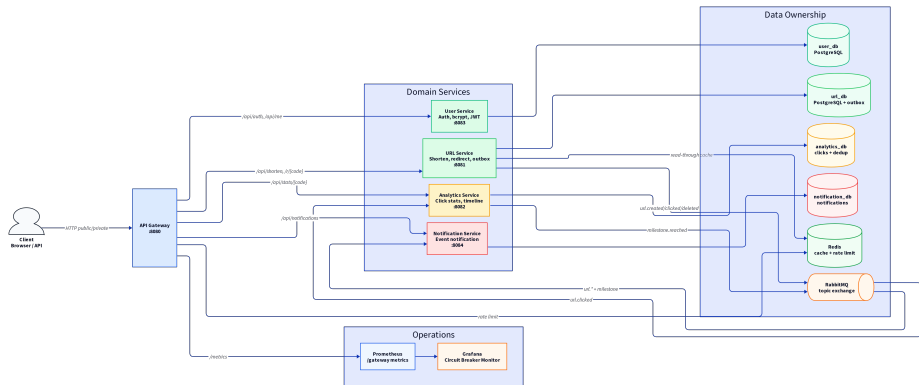


Roadmap này bám đúng đề cương: config/secrets, tracing, CI/CD, cloud/Kubernetes, contract test và self-healing.

Checklist phòng thủ câu hỏi nhanh

Câu hỏi	Trả lời ngắn
Vì sao là microservice?	Binary/container/DB riêng, HTTP + RabbitMQ, không shared DB.
Redis chết thì sao?	Fallback PostgreSQL; Redis không phải source of truth.
Analytics chết thì redirect có chết không?	Không, click đi qua outbox/RabbitMQ, xử lý eventual consistency.
Có exactly-once không?	Không hứa; at-least-once + dedup ở analytics.
URL service chết thì sao?	Gateway circuit breaker OPEN và trả 503 nhanh.
Có chống spam không?	Rate limit theo IP bằng Redis ở gateway.
Có bảo mật password không?	bcrypt, không lưu plaintext, JWT có expiry.
Có quan sát hệ thống không?	Prometheus metrics, Grafana dashboard, structured logs, correlation ID.

D2 Architecture Map: nhìn toàn hệ thống trong một hình



Sơ đồ D2 này cố tình thể hiện cả runtime boundary, data ownership và operations để chứng minh đây là một hệ thống microservices có vận hành, không chỉ là nhiều package Go.

Rubric SE361: mapping từng tiêu chí vào dự án

Tiêu chí môn học	Hiện thực trong dự án	Slide/demo để bảo vệ
DDD, SRP, Bounded Context	User, URL, Analytics, Notification, Gateway tách trách nhiệm rõ	Service responsibility + DB ownership
RESTful API	Gateway expose endpoint public/private, service nội bộ HTTP/JSON	API surface, route table
Async communication	RabbitMQ topic exchange, durable queue, event contract	Event pipeline D2
Database per Service	4 PostgreSQL DB riêng, không shared schema	Database per service table
Resilience	Rate limit, timeout, circuit breaker, fail-open Redis limiter	CB state machine + load test
Observability	JSON logs, correlation ID, Prometheus, Grafana	Deployment/observability D2
Testing	Unit, smoke, e2e, k6 load test	Testing matrix

Từ monolith sang microservices: quyết định thiết kế

Monolith ngây thơ

Auth, shorten, redirect, analytics, notification cùng process và cùng DB. Dễ demo nhưng khó scale và dễ cascading failure.

Tách service

Tách bounded context theo nghiệp vụ, service sở hữu schema riêng, chỉ giao tiếp qua API/event.

Vận hành được

Gateway bảo vệ edge, RabbitMQ hấp thụ side effect, Redis giảm tải hot path, Prometheus/Grafana quan sát lỗi.

Luận điểm: URL shortener là bài toán nhỏ nhưng có profile tải rất thực tế: redirect đọc nhiều, analytics ghi nhiều, notification là side effect; vì vậy microservices có lý do kỹ thuật, không phải tách cho có.

API

- ▶ POST /register: tạo user.
- ▶ POST /login: phát JWT.
- ▶ GET /me: đọc identity từ token.
- ▶ GET /health: healthcheck container.

Chi tiết đăng kể

- ▶ Email validate bằng regex và unique constraint.
- ▶ Password tối thiểu 8 ký tự, hash bằng bcrypt.
- ▶ JWT có issuer, issued-at, expiry, subject, email.
- ▶ Body register/login giới hạn 1MB.
- ▶ Có fake verify khi user không tồn tại để giảm timing leak.

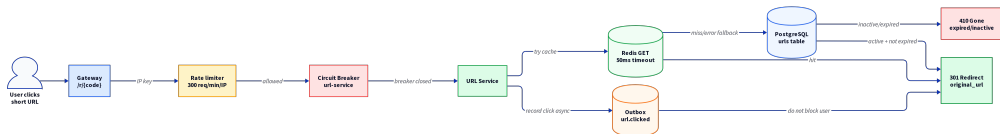
Domain commands

- ▶ Shorten URL: validate scheme, generate code, persist.
- ▶ List URLs: cursor pagination, limit clamp 1..100.
- ▶ Delete URL: ownership check, soft deactivate, cache invalidation.
- ▶ Redirect: cache first, DB fallback, status 410 nếu expired/inactive.

Domain events

- ▶ `url.created`: notification biết có link mới.
- ▶ `url.clicked`: analytics ghi click.
- ▶ `url.deleted`: notification biết link bị xóa.
- ▶ Event ghi vào outbox cùng transaction với URL mutation.

D2 Redirect Hot Path: nơi chịu tải cao nhất



Điểm cần nói khi demo: redirect không chờ analytics consumer. Click event được ghi async; người dùng nhận redirect nhanh nhất có thể.

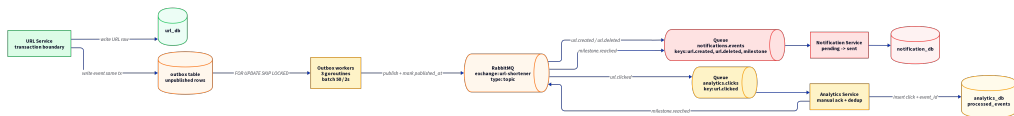
Thiết kế mã ngắn

- ▶ Alphabet base62: a-zA-Z0-9.
- ▶ Độ dài 7 ký tự: $62^7 \approx 3.52$ nghìn tỷ mã.
- ▶ Sinh bằng `crypto/rand`, không dùng `math/rand`.
- ▶ PostgreSQL unique constraint là hàng rào cuối.

Collision handling

- ▶ Nếu insert đụng unique `short_code`, service retry.
- ▶ Retry tối đa 3 lần để tránh loop vô hạn.
- ▶ Với 1 triệu URL, xác suất collision cực thấp theo dev docs.
- ▶ Không cần coordination service để cấp mã trong scope đồ án.

D2 Event Pipeline: Outbox + RabbitMQ + Consumers



RabbitMQ Topology: topic exchange có chủ đích

Exchange

- ▶ Name: `url-shortener`.
- ▶ Type: `topic`.
- ▶ Durable để queue/exchange sống qua restart.
- ▶ Routing key mô tả domain event.

Bindings

- ▶ `analytics.clicks` nhận `url.clicked`.
- ▶ `notifications.events` nhận `url.created`, `url.deleted`, `milestone.reached`.
- ▶ Analytics phát ngược `milestone.reached` khi đạt 10/100/1000 clicks.
- ▶ Consumer không gọi chéo để hỏi thêm dữ liệu nếu event đã đủ payload.

Data Consistency: chấp nhận eventual consistency có kiểm soát

Điều không làm

- ▶ Không distributed transaction 2PC.
- ▶ Không shared DB để join trực tiếp.
- ▶ Không hứa exactly-once toàn hệ thống.
- ▶ Không để redirect gọi sync sang analytics/notification.

Điều đã làm

- ▶ Outbox đảm bảo DB mutation và event cùng transaction.
- ▶ At-least-once delivery qua RabbitMQ.
- ▶ Analytics dedup bằng `processed_events`.
- ▶ Milestone unique constraint tránh bắn nhiều lần.

Schema Design: chi tiết để tránh bị hỏi hổng

DB	Bảng chính	Lý do thiết kế
user_db	users(id, email, password_hash, created_at)	User service sở hữu identity; email unique để chống duplicate.
url_db	urls, outbox	URL là source of truth; outbox là cầu nối sang messaging.
analytics_db	clicks, processed_events, milestones	Click append-only, event dedup, milestone unique.
notification_db	notifications(user_id, payload, status, sent_at)	Side effect được lưu trạng thái để debug/retry.

Indexes cần nhắc: short_code cho redirect, user_id + created_at cho list URL, outbox unpublished cho worker, processed_events(event_id) cho dedup.

Analytics: không chỉ đếm tổng click

Counters

Total clicks, last 24h, last 7d. Các query chạy song song để giảm latency API stats.

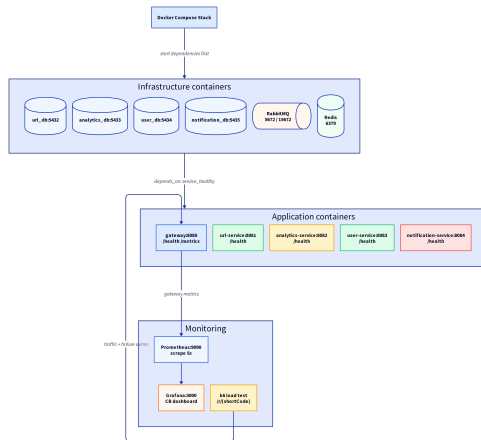
Breakdown

Top referers, timeline theo day/hour, lưu user-agent và referer từ click event.

Milestone

Khi đạt 10, 100, 1000 clicks, analytics phát `milestone.reached` cho notification.

D2 Deployment & Observability: chạy được cả stack



Docker Compose: thứ tự khởi động và healthcheck

Infrastructure first

- ▶ 4 PostgreSQL containers có `pg_isready`.
- ▶ RabbitMQ dùng `rabbitmq-diagnostics ping`.
- ▶ Redis dùng `redis-cli ping`.
- ▶ Volumes riêng cho DB, RabbitMQ, Redis, Prometheus, Grafana.

App depends on healthy deps

- ▶ Service chỉ start khi DB/broker/cache healthy.
- ▶ Mỗi app expose `/health`.
- ▶ Gateway phụ thuộc 4 domain service healthy.
- ▶ Smoke test kiểm tra ports 8080–8084.

Prometheus metrics: không chỉ có log

Gateway metrics

- ▶ gateway_requests_total
- ▶ gateway_request_duration_seconds
- ▶ gateway_circuit_breaker_state
- ▶ gateway_circuit_breaker_trips_total
- ▶ gateway_circuit_breaker_rejected_total

Grafana panels

- ▶ Circuit Breaker State: 0/1/2.
- ▶ Requests/sec by status class.
- ▶ Error Rate %.
- ▶ CB Trips total.

Load Test Script: thuyết phục bằng quy trình demo

Happy path

- 1 `docker compose up -d -build`
- 2 `./scripts/smoke_test.sh`
- 3 `k6 run scripts/load_test.js`
- 4 `k6 register, login, shorten URL.`
- 5 Traffic tập trung vào `/r/{shortCode}`.

Failure path

- 1 Đang load test thì stop url-service.
- 2 Gateway bắt đầu connection error.
- 3 Sau 5 lỗi trong 10s: breaker OPEN.
- 4 Request mới trả 503 nhanh, không chờ timeout downstream.
- 5 Start lại service, 30s sau HALF_OPEN probe.

Security Threat Model ngắn

Rủi ro	Cơ chế hiện tại	Hạn chế / mở rộng
Password leak	bcrypt hash, không lưu plaintext	Cần secret rotation, password policy mạnh hơn
JWT tampering	Verify HMAC signing method, issuer, expiry	Chưa có refresh/revocation
URL abuse/spam	Rate limit theo IP, body limit auth	Có thể thêm per-user quota và CAPTCHA
Privacy analytics	Hash IP trước khi lưu event	Salt config cần audit kỹ
Unauthorized delete	Ownership check bằng user id	Có thể thêm RBAC/team ownership
Secret exposure	Env config cho demo	Production cần Vault/K8s Secret

Những câu hỏi khó và cách trả lời không bị hớ

Câu hỏi khó	Câu trả lời nên dùng
Nếu RabbitMQ down lúc tạo URL?	URL và event vẫn nằm trong DB/outbox; worker publish lại khi broker phục hồi.
Nếu consumer xử lý trùng event?	Analytics dedup bằng <code>processed_events</code> ; milestone có unique constraint. Notification duplicate là hạn chế đã nhận diện.
Vì sao không Saga?	Use case không cần distributed transaction nhiều bước có compensation; Outbox + eventual consistency đủ phù hợp.
Redis cache stale sau delete?	Delete URL set inactive trong DB và xóa cache Redis; DB vẫn là source of truth.
Tại sao gateway verify JWT mà service vẫn cần biết user?	Gateway là lớp edge; downstream vẫn nhận identity qua header/context để enforce ownership.
Có scale thật chưa?	Compose demo một replica; thiết kế stateless cho gateway/url-service và cache/event broker hỗ trợ scale ngang.

URL Shortener Microservices

Một case study nhỏ nhưng đủ lát cắt của môn SE361: DDD, database-per-service, REST, messaging, event-driven, outbox, cache, JWT, rate limit, circuit breaker, monitoring, load testing và Docker deployment.

Scalable

cache + async + stateless
edge

Resilient

rate limit + circuit breaker
+ outbox

Observable

logs + metrics + Grafana